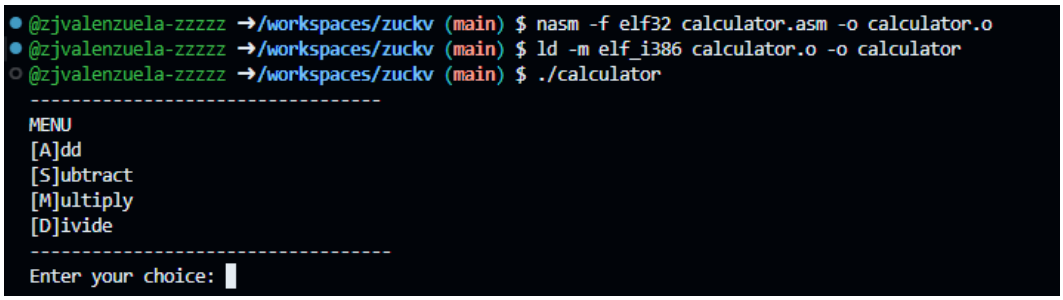
Zack Gabriel J. Valenzuela
TN24

LABEXER1 ASSEMBLY LANGUAGE

COMPILATION



EXPLANATION:

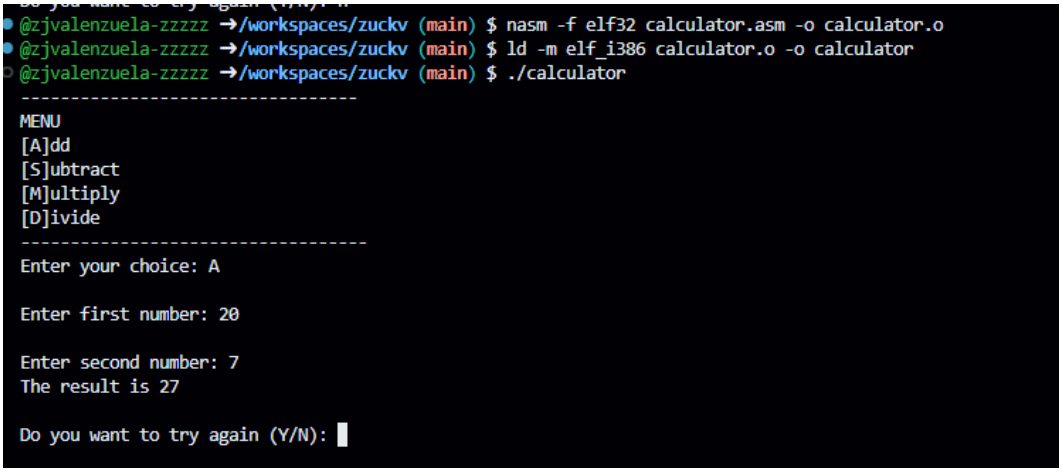
When you run the program, it clears your terminal screen and displays a clean menu showing the four choices: Add, Subtract, Multiply, and Divide. It asks you to type your choice, and then prompts you to enter a first number and a second number one by one.

As you type your numbers, the program carefully checks your input. If you try to enter a negative number or if you try to subtract a larger number from a smaller one the program instantly catches it, skips the calculation, and prints your error message: "Invalid Number Please input a positive number".

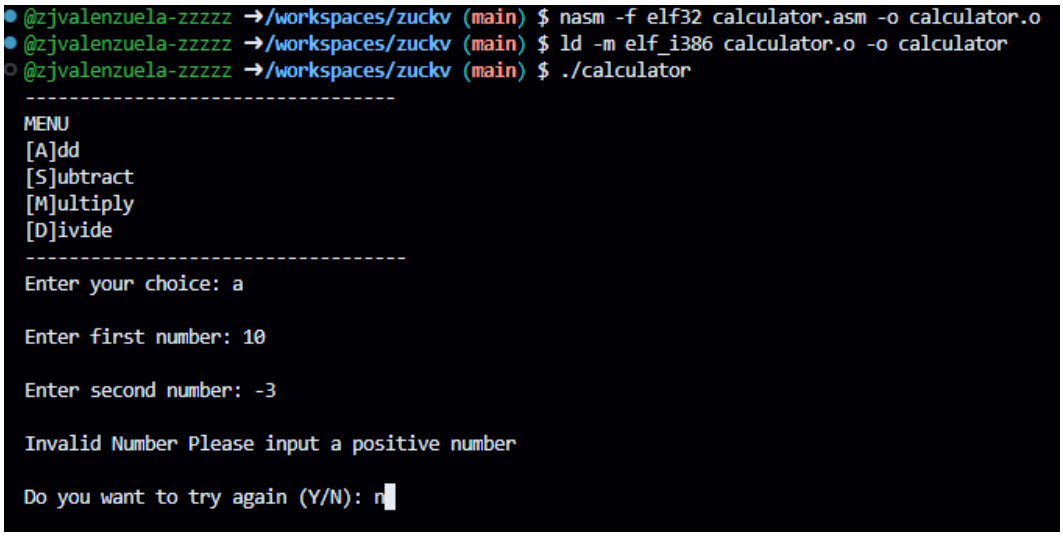
If your numbers are valid positive numbers, the program performs the math you selected. For division, it even calculates and displays any leftover remainders. After showing the result, the program asks if you want to try again. If you type 'Y', the screen clears itself and loops back to the main menu so you can calculate something else; if you type any other key, the program safely shuts down and exits.

TEST CASES 1 (ADDITION)

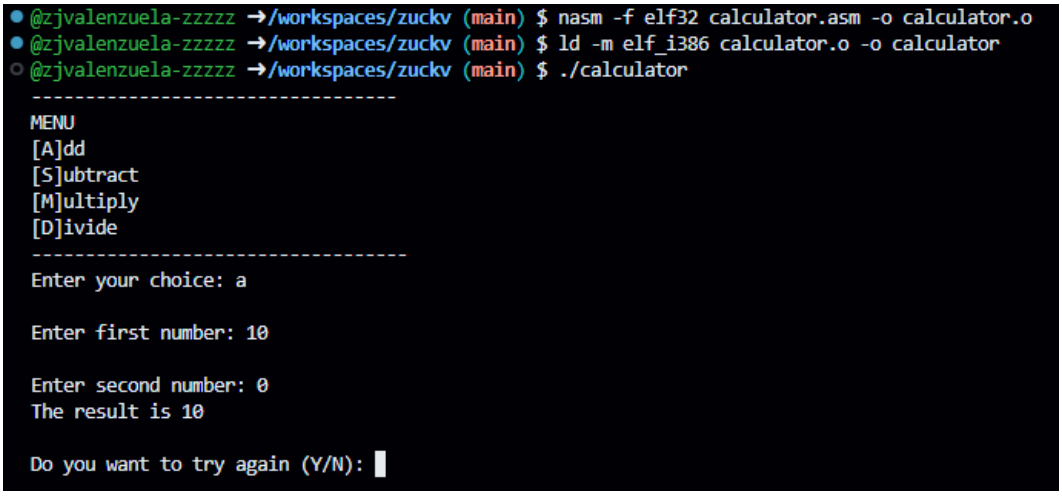
#1 (normal run)



#2 (negative input)



#3(addition with zero)



TEST CASES 2(SUBTRACTION)

#1 (normal run)

```
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ nasm -f elf32 calculator.asm -o calculator.o
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ld -m elf_i386 calculator.o -o calculator
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ./calculator
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: s

Enter first number: 20

Enter second number: 7
The result is 13

Do you want to try again (Y/N):
```

#2 (negative input)

```
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ nasm -f elf32 calculator.asm -o calculator.o
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ld -m elf_i386 calculator.o -o calculator
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ./calculator
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: s

Enter first number: -5

Invalid Number Please input a positive number

Do you want to try again (Y/N):
```

#3 (subtraction underflow / negative result)

```
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ nasm -f elf32 calculator.asm -o calculator.o
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ld -m elf_i386 calculator.o -o calculator
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $ ./calculator
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: s

Enter first number: 10

Enter second number: 15

Invalid Number Please input a positive number

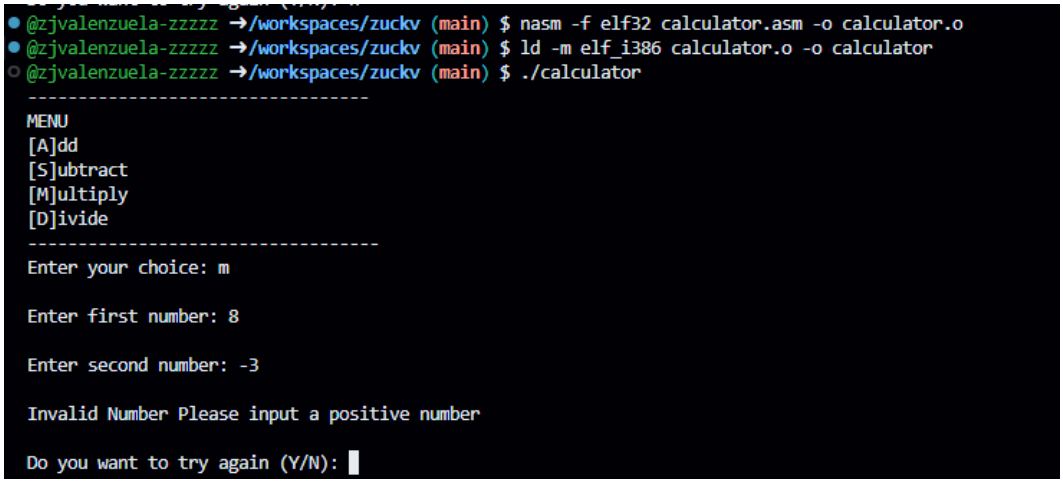
Do you want to try again (Y/N): n
@zjvalenzuela-zzzzz ➜/workspaces/zuckv (main) $
```

TEST 3 (MULTIPLICATION)

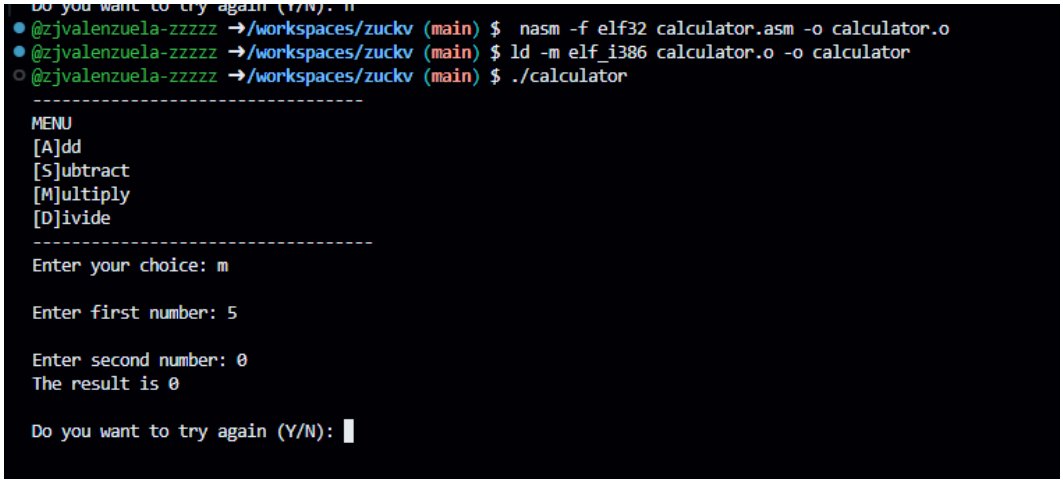
#1 (normal run)



#2 (negative input)

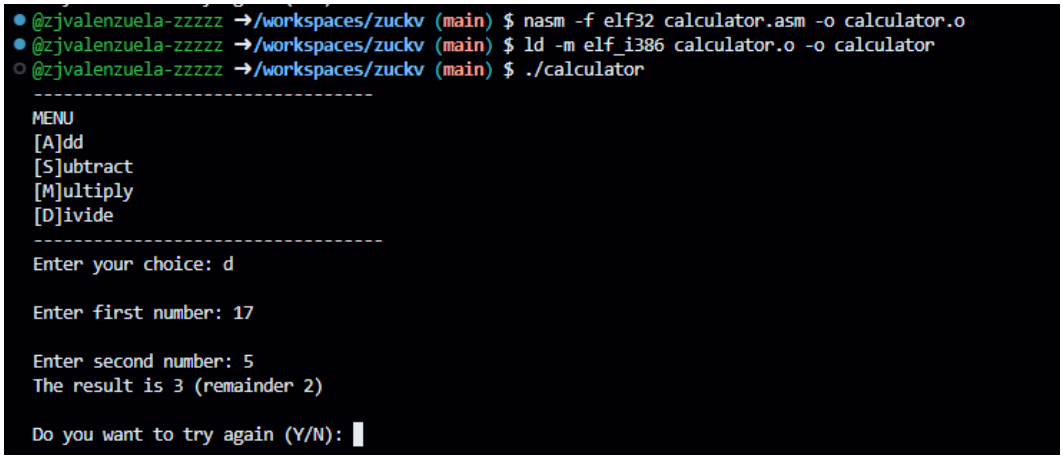


#3 (multiplication by zero)

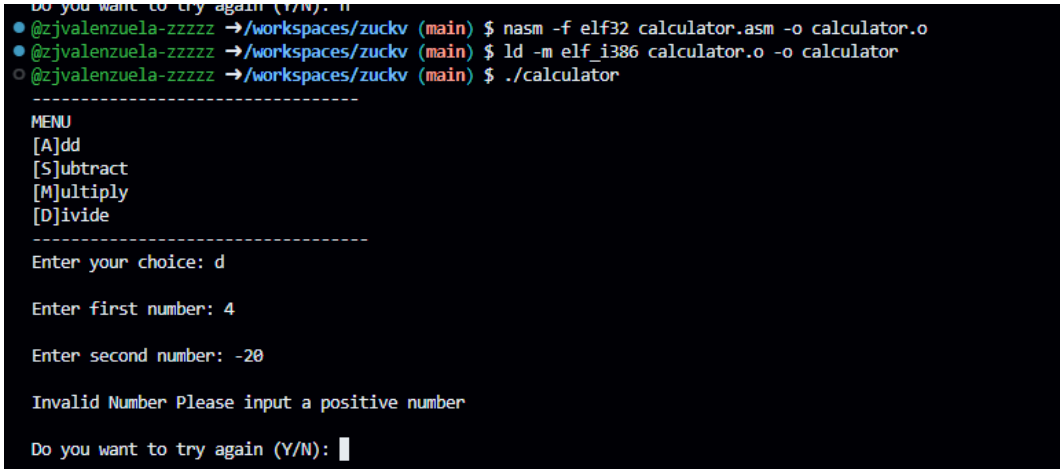


TEST CASE 4 (DIVISION)

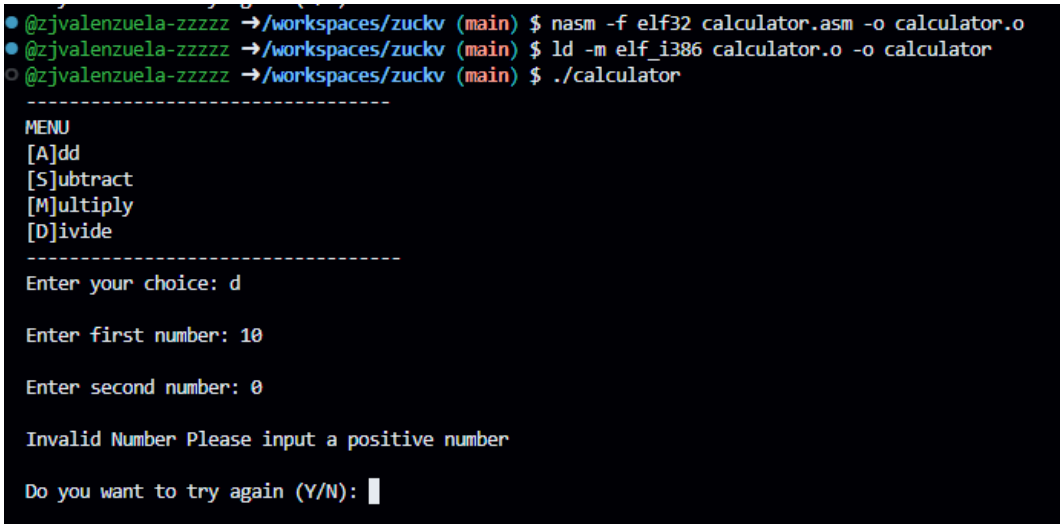
#1 (normal run with a remainder)



#2 (negative input)



#3 (division by zero)



SOURCE CODE:

```
; ZACK GABRIEL J. VALENZUELA
; NASM32 CALCULATOR

section .data
   CLR       db 27, "[2J", 27, "[1;1H", 0
   MENU      db "---------------------------------", 10
             db "MENU", 10
             db "[A]dd", 10
             db "[S]ubtract", 10
             db "[M]ultiply", 10
             db "[D]ivide", 10
             db "---------------------------------", 10
             db "Enter your choice: ", 0
   P1        db 10, "Enter first number: ", 0
   P2        db 10, "Enter second number: ", 0
   RES       db "The result is ", 0
   REM       db " (remainder ", 0
   ERR       db 10, "Invalid Number Please input a positive number", 10, 0
   AGAIN     db 10, "Do you want to try again (Y/N): ", 0

section .bss
   buf       resb 32
```

```
    num1      resd 1
    num2      resd 1
    op        resb 2

section .text
    global _start

_start:
menu:
    mov esi, CLR
    call print
    mov esi, MENU
    call print
    call read
    mov al, [buf]
    and al, 0xDF
    mov [op], al

    mov esi, P1
    call print
    call read
    call atoi
    jc err_out
    mov [num1], eax

    mov esi, P2
    call print
    call read
    call atoi
    jc err_out
    mov [num2], eax

    mov eax, [num1]
    mov ebx, [num2]
    mov cl, [op]

    cmp cl, 'A'
    je do_add
    cmp cl, 'S'
    je do_sub
    cmp cl, 'M'
    je do_mul
    cmp cl, 'D'
    je do_div
```

```
    jmp retry

do_add:
    add eax, ebx
    jmp print_res
do_sub:
    sub eax, ebx
    jl err_out      ; prevent negative result / below zero
    jmp print_res
do_mul:
    imul eax, ebx
    jmp print_res
do_div:
    cmp ebx, 0
    je err_out
    xor edx, edx
    div ebx         ; eax = quotient, edx = remainder
    push edx        ; Save remainder on stack
    push eax        ; Save quotient on stack

    mov esi, RES
    call print

    pop eax
    call print_num

    pop edx
    cmp edx, 0
    je done_div

    push edx
    mov esi, REM
    call print
    pop eax
    call print_num
    mov byte [buf], ')'
    mov byte [buf+1], 10
    mov byte [buf+2], 0
    mov esi, buf
    call print
done_div:
    jmp retry

print_res:
```

```
    push eax
    mov esi, RES
    call print
    pop eax
    call print_num
    mov byte [buf], 10
    mov byte [buf+1], 0
    mov esi, buf
    call print
    jmp retry

err_out:
    mov esi, ERR
    call print

retry:
    mov esi, AGAIN
    call print
    call read
    mov al, [buf]
    and al, 0xDF
    cmp al, 'Y'
    je menu

    mov eax, 1
    xor ebx, ebx
    int 0x80

; --- HELPERS ---
print:
    mov edx, esi
.l: cmp byte [edx], 0
    je .p
    inc edx
    jmp .l
.p: sub edx, esi
    mov ecx, esi
    mov eax, 4
    mov ebx, 1
    int 0x80
    ret

read:
    mov eax, 3
```

```
    mov ebx, 0
    mov ecx, buf
    mov edx, 32
    int 0x80
    ret

atoi:
    xor eax, eax
    mov esi, buf
.l: mov bl, [esi]
    cmp bl, 10      ; newline check
    je .done
    cmp bl, 13      ; carriage return check
    je .done
    cmp bl, 0       ; null terminator check
    je .done
    cmp bl, '-'     ; reject negative sign
    je .err
    cmp bl, '0'
    jl .err
    cmp bl, '9'
    jg .err
    sub bl, '0'
    imul eax, eax, 10
    movzx ebx, bl
    add eax, ebx
    inc esi
    jmp .l
.err:
    stc             ; set carry flag to signal error
    ret
.done:
    clc             ; clear carry flag on success
    ret

print_num:
    mov edi, buf + 31
    mov byte [edi], 0
    mov ebx, 10
    test eax, eax
    jnz .c
    dec edi
    mov byte [edi], '0'
    jmp .p
```

```
.c: xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .c
.p: mov esi, edi
    call print
    ret
```